Instabilité de NegGrad+ dans les réseaux à BatchNorm

Quand l'ensemble à oublier est concentré dans une seule classe

Khalil Bibi

Octobre 2026

Table des matières

Objectif **2**

1 Cadre expérimental **2**
1.1 Données et notations . . . 2
1.2 Modèle et entraînement de O . . . 2
1.3 Initialisation des poids . . . 2
1.4 BatchNorm . . . 3
1.5 Formats numériques . . . 3
1.6 NegGrad+ et la procédure d'oubli . . . 4
1.7 Mesures . . . 4
1.8 Comment on juge l'oubli . . . 4

2 Étape 1 : étude de départ et diagnostics **5**
2.1 L'étude de départ . . . 5
2.2 Diagnostic 1 : la mémoire de BatchNorm ou les poids ? . . . 5
2.3 Diagnostic 2 : comment le lot d'oubli passe dans BatchNorm . . . 6
2.4 Diagnostic 3 : quelle partie de BatchNorm ? . . . 6

3 Étape 2 : réplication avec des tests statistiques **7**
3.1 Plan . . . 7
3.2 La statistique U . . . 8
3.3 Résultats . . . 9
3.4 Particularités de CIFAR-10 / ResNet-18 . . . 9

4 Étape 3 : quelle partie de BatchNorm ? **10**
4.1 Plan . . . 10
4.2 Mesures . . . 10
4.3 Résultats . . . 11

5 Étape 4 : concentration ou classes difficiles ? **11**
5.1 Plan . . . 11
5.2 Mesure : la perte précoce . . . 12
5.3 Modèle . . . 12
5.4 Résultats . . . 12

6 Étape 5 : est-ce l'objectif de NegGrad+ ? **13**
6.1 Premier essai : la charge d'écrêtage . . . 13
6.2 Deuxième essai : la taille typique des gradients $\bar{G}$ . . . 14

7 Étape 6 : des remèdes à l'instabilité **15**
7.1 Comment on mesure les dégâts 15
7.2 Remède principal : le réglage par condition 15
7.3 Les deux autres remèdes 16

8 Limites et perspectives **16**

Objectif

On essaye de montrer que **NegGrad+ devient instable dans les réseaux à BatchNorm quand l'ensemble à oublier est concentré dans une seule classe**, alors qu'un ensemble **stratifié** de même taille ne pose pas de problème. Ensuite on cherche quelle partie de BatchNorm est en cause, si l'objectif de NegGrad+ y est pour quelque chose, et si on peut réparer.

1 Cadre expérimental

1.1 Données et notations

— **Fashion-MNIST** : 60 000 images d'entraînement (28×28, niveaux de gris, 10 classes). On met 5 000 images de côté pour la validation (500 par classe), donc $|D| = 55\,000$. Les 10 000 images de test ne servent qu'à l'évaluation.

— D_f : l'ensemble à oublier, toujours **600 images**. $D_r = D \setminus D_f$: l'ensemble à garder. Donc $D = D_f \cup D_r$.

— **Concentré** (c0 à c9) : les 600 images viennent d'une seule classe, soit environ 11 % de la classe. Le reste de la classe reste dans D_r : on veut oublier *ces* 600 bottines, pas la notion de bottine.

— **Stratifié** : 60 images de chaque classe.

— **Modèle de référence** R : le modèle réentraîné depuis zéro sur D_r seul, avec la même recette que O. C'est ce dont on veut se rapprocher sans devoir réentraîner. (On verra qu'à partir de l'étape 2, R sert surtout de repère pour juger l'oubli, pas pour mesurer l'instabilité.)

1.2 Modèle et entraînement de O

On utilise un petit réseau de neurones à convolutions : 3 blocs Conv 3×3 (sans biais) + BatchNorm + ReLU, avec 32, 64 puis 128 filtres et deux MaxPool, puis deux couches denses (256 puis 10). Ça fait environ 1,7 M de paramètres et 3 couches BatchNorm.

— SGD, learning rate 0,05 en cosinus, momentum 0,9, weight decay 5×10^{-4} ;

— 15 époques, batch de 128, sans augmentation ;

— environ **93 %** de précision sur le test. Un modèle O par graine.

1.3 Initialisation des poids

Dans toutes nos expériences on utilise les **initialisations par défaut de PyTorch** : on n'a écrit aucune initialisation (aucun `nn.init` ni `kaiming` dans les notebooks).

— **Convolutions et couches denses** : chaque poids est tiré uniformément dans $[-b, b]$ avec $b = 1/\sqrt{n}$, où n est le nombre d'entrées du neurone. Ça donne une variance $\mathrm{Var}(w) = b^2/3 = 1/(3n)$.

— **Biais** : les convolutions n'en ont pas (le β de la BatchNorm qui suit joue ce rôle) ; ceux des couches denses sont aussi tirés uniformément dans $[-1/\sqrt{n}, 1/\sqrt{n}]$.

— **BatchNorm** : les paramètres appris partent de $\gamma = 1$ et $\beta = 0$; les buffers partent d'une moyenne mémorisée de 0 et d'une variance mémorisée de 1.

Pourquoi la variance compte. Si la variance du signal explose d'une couche à l'autre, on se retrouve avec des logits énormes en sortie, donc une classe à 100 % après le softmax, et des pertes et des gradients énormes. Si elle tend vers 0, tous les logits sont proches de 0, donc on a environ 10 % sur chaque classe et des gradients minuscules.

Kaiming (He). Kaiming prend $\mathrm{Var}(w) = 2/n$ (le 2 compense la moitié du signal que la ReLU met à zéro), ce qui garde la même variance d'une couche à l'autre. Le défaut de PyTorch donne $1/(3n)$, soit 6 fois moins : sans normalisation, la variance baisserait d'environ $\times 6$ par couche.

Pourquoi ça ne pose pas de problème ici. Chaque convolution est suivie d'une BatchNorm qui remet l'échelle à 1, donc le rétrécissement ne s'accumule pas ; seules les deux couches denses finales le subissent. C'est pareil pour le ResNet-18 de l'étape 2 : aucune initialisation spéciale, y compris autour des skip connections.

Graines. La graine est fixée juste avant de construire le modèle, donc même graine = mêmes poids de départ au bit près. Les méthodes d'oubli partent d'une **copie de** O ; chaque R a sa propre graine d'initialisation.

1.4 BatchNorm

BatchNorm travaille **par canal**. Pour un canal c, on calcule la moyenne μ_B et la variance σ_B^2 sur **toutes les images du lot et toutes les positions** ($N \times H \times W$ valeurs), puis :

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \qquad y = \gamma\,\hat{x} + \beta$$

où γ et β sont appris par le gradient (ils permettent au réseau de réétirer et redécaler les valeurs, voire d'annuler la normalisation si ça l'arrange).

— **Mode train** : on normalise avec les statistiques du lot courant, et le gradient passe à travers μ_B et σ_B^2. En plus, on met à jour les buffers à chaque lot :

$$\mu_{\text{mém}} \leftarrow 0{,}9\,\mu_{\text{mém}} + 0{,}1\,\mu_B \qquad \text{(idem pour la variance)}$$

Les buffers ne sont *pas* appris : ils sont mesurés.

— **Mode eval** : on normalise avec les buffers, qui ne bougent plus, puis on applique γ et β. La couche devient une transformation affine fixe par canal.

— **Recalibration** : on recalcule exactement les buffers sur tout D_r (vraie moyenne et vraie variance), couche par couche, sans toucher aux poids ni à γ et β. Les images à oublier n'y servent jamais.

1.5 Formats numériques

Tout l'entraînement est en **FP32**, il n'y a pas de demi-précision. Sur A100, PyTorch calcule par défaut les convolutions en **TF32** (arrondi interne à environ 3 chiffres), d'où des tolérances de contrôle de 10^{-5} par couche et 10^{-4} en relatif sur le réseau. Les vérifications sensibles (buffers, normes de gradient) sont faites en **FP64**. Le FP16 n'a servi qu'à *enregistrer* certaines mesures descriptives dans l'étude de l'étape 3 (un dépassement de 65 504 a été corrigé par amendement).

1.6 NegGrad+ et la procédure d'oubli

On cherche à oublier D_f avec la perte suivante :

$$L = \alpha \, \mathrm{CE}_r - (1 - \alpha) \, \mathrm{CE}_f$$

On fait *baisser* la perte sur D_r et *monter* la perte sur D_f. Comme $\mathrm{CE} = -\log p$ n'a pas de plafond (quand $p \to 0$, $-\log p \to \infty$), rien n'arrête la poussée.

- Une époque sur D_r = **425 pas**. À chaque pas : un lot de **128** images de D_r et un lot de **64** images de D_f (on enchaîne des permutations de D_f, donc chaque image est vue environ 45 fois).
- SGD, lr 0,01 constant, momentum 0,9, weight decay 5×10^{-4}.
- Poids d'oubli $1 - \alpha \in \{0{,}001; 0{,}01; 0{,}05\}$, 0,01 en principal.
- **Clipping** de la norme globale du gradient à 5 : une ceinture de sécurité qui évite des pas gigantesques quand CE_f s'emballe.

1.7 Mesures

Mesure	**Définition**	**Remarque**
Divergence	$\mathrm{CE}_f > 10$ à un pas, ou échec numérique (NaN, inf)	$\mathrm{CE}_f > 10$ veut dire que la proba moyenne (géométrique) de la bonne classe sur le lot est $< e^{-10} \approx 0{,}005\,\%$. Le hasard donne $\mathrm{CE} = \ln 10 \approx 2{,}3$.
Effondrement	Précision test $\leq 20\,\%$, ou $\geq 80\,\%$ des prédictions dans une seule classe	Mesuré avec les buffers natifs (le modèle tel quel).
S_{moy}	Moyenne sur les pas de $\log_{10} \max(\mathrm{CE}_f, 10^{-7})$	Plus S_{moy} est grand, plus le run est instable.

1.8 Comment on juge l'oubli

On ne peut jamais vérifier directement qu'un modèle a oublié : un modèle a oublié D_f s'il est impossible à distinguer de R, le modèle qui n'a jamais vu D_f. Et oublier ne veut pas dire se tromper : en concentré, R reconnaît encore $\sim$93 % des images à oublier, parce qu'il a appris la classe avec les autres images. Un modèle qui tombe bien plus bas en fait trop, et ça se repère aussi. Dans ce rapport on utilise trois repères, toujours avec des buffers recalibrés :

- **la précision sur D_f, comparée à R** : au-dessus de R le modèle n'en fait pas assez, bien en dessous il casse ;
- **la force** E = moyenne sur D_f de $\log_{10} p_O(y \mid x) - \log_{10} p_{\text{modèle}}(y \mid x)$: de combien la confiance dans la bonne classe a baissé par rapport à O, en décades ; E_R est la même chose pour R ;
- **une attaque d'appartenance** (une seule fois, exploratoire, sur Tiny ImageNet) : un attaquant essaie de deviner si une image faisait partie de l'entraînement ; une AUC de 0,5 veut dire qu'il n'y arrive pas.

Ces repères servent à vérifier qu'une méthode qui ne casse pas fait quand même quelque chose. Le rapport ne prétend jamais qu'un modèle a *effacé* D_f.

2 Étape 1 : étude de départ et diagnostics

Attention

Ces analyses sont **post hoc** : les classes et les variantes ont été choisies après avoir vu l'étude de départ. Elles servent à formuler des hypothèses, qu'on teste ensuite sur de nouveaux modèles (étapes 2 à 4).

2.1 L'étude de départ

On a fait, sur 5 graines, 11 runs : 1 par classe (10 ensembles concentrés) et 1 stratifié, avec 600 images à oublier. Ça fait **55 runs NegGrad+**.

Ensemble	Divergences (/5)	Effondrements (/5)
c1, c2, c4, c6	5	5
c5	4	3
c7	5	3
c0, c3, c8, c9	0	0
Stratifié	0	0
Total	**29 / 55**	**26 / 55**

Lecture

On a 26 runs sur 55 qui s'effondrent (précision finale $\leq 20\,\%$) et 29 qui divergent. Mais ça ne concerne **que les ensembles concentrés** (0 fois en stratifié), et toujours les mêmes six classes : 1, 2, 4, 6, et parfois 5 et 7. Ce n'est donc pas de l'oubli, c'est une instabilité. On essaye du coup de comprendre ce qui se passe dans BatchNorm.

2.2 Diagnostic 1 : la mémoire de BatchNorm ou les poids ?

On reprend les mêmes graines et on rejoue les 55 runs **bit à bit** (même O, mêmes lots, même ordre), en ne changeant que la gestion des buffers. Dans la version d'origine **V0**, à chaque pas :

```
1. sortie_r = réseau(lot_garder)  // BN : stats du lot à garder, met à jour ses
buffers
2. sortie_f = réseau(lot_oubli)   // BN : stats du lot à oublier, met à jour ses
buffers
3. perte = α·CE(sortie_r) − (1−α)·CE(sortie_f)
4. une seule backward sur cette perte, clipping à 5, puis mise à jour des poids
```

— **V1c** : à la ligne 2, le lot à oublier ne met plus à jour les buffers.

— **V2** : après l'oubli V0, on recalibre les buffers sur tout D_r.

— $\alpha = 1$: même procédure sans terme d'oubli, pour voir si l'entraînement seul peut causer l'effondrement.

Version	Précision test (sur les 26 runs effondrés)
V0	10 % (médiane)
V1c	69 % (médiane)
V2 (recalibré)	74 % (médiane)
$\alpha = 1$	jamais d'effondrement (minimum 91 %)

Lecture

Le terme d'oubli est nécessaire ($\alpha = 1$ ne casse rien). Le buffer pollué par les lots d'oubli explique une grosse partie de l'effondrement mesuré, mais pas tout : même recalibrés, les modèles restent vers 74 % au lieu de 93 %, donc **les poids sont aussi abîmés**. Sur c1, CE_f monte jusqu'à 1,9 à 3,0 $\times 10^5$, avec 68 à 74 % des pas clippés.

2.3 Diagnostic 2 : comment le lot d'oubli passe dans BatchNorm

On reprend les mêmes 5 graines et les 6 classes qui s'effondrent, donc **30 runs par variante** ($1 - \alpha = 0{,}01$), et on compte les divergences :

— **Variante A** : la même version que V0.

— **Variante B** : on fait passer le lot à garder et le lot à oublier ensemble :

```
1. sortie = réseau(lot_garder + lot_oubli)  // BN : stats du lot combiné (128 + 64),
                                            // met à jour ses buffers
2. sortie_r = sortie[0:128], sortie_f = sortie[128:]
3. perte = α·CE(sortie_r) − (1−α)·CE(sortie_f)
4. une seule backward sur cette perte, clipping à 5, puis mise à jour des poids
```

— **Variante C** : on garde les BatchNorm en mode évaluation (avec les buffers initiaux de O), tout en mettant à jour γ et β pendant la backward.

Variante	Divergences	Précision test	Précision sur D_f (c6)	
A	29 / 30	effondrée	–	
B	0 / 30	à ∼1,5 point de O	94,5 %	(R : 79,5 %)
C	0 / 30	à ∼1,5 point de O	93,0 %	(R : 79,5 %)

Lecture

B et C font passer de 29 divergences à 0. Ça indique que **la normalisation du lot d'oubli par ses propres statistiques mono-classe** (et le gradient qui passe à travers) joue un rôle dans les divergences pendant l'entraînement. Attention : la divergence se mesure en mode train, où les buffers ne servent pas à normaliser, donc ce n'est pas la pollution des buffers qui est en cause ici. Et c'est un indice, pas une preuve : B et C changent plusieurs choses à la fois.
Par contre l'oubli est très faible : sur les images à oublier, B et C restent proches de O, pas de R. Ils se comportent presque comme du fine-tuning : le modèle reste très bon pour reconnaître ces images.

2.4 Diagnostic 3 : quelle partie de BatchNorm ?

On regarde maintenant ce qui contribue à l'instabilité en bidouillant BatchNorm de plusieurs manières : est-ce que μ et σ^2 sont des constantes dans la backward ou pas, avec quoi on normalise le lot à oublier, et est-ce que le lot à oublier met à jour les buffers. On prend **c1** (qui diverge) et **c8** comme **contrôle stable** (0 divergence sous A), avec 5 graines, donc 10 runs par variante. La passe du lot à garder ne change pas ; seule la passe du lot à oublier change :

Variante	Lot d'oubli normalisé par	Gradient à travers les stats	Met à jour les buffers
A	ses propres stats	oui	oui
E2	ses propres stats	oui	non
E1D	ses propres stats	non (détachées)	non
E1	les buffers courants (mis à jour par le lot à garder seul)	non	non
E1-O	les buffers figés de O	non	non

Variante	Div. c1	CE_f max (c1)	Précision test c1	Div. c8
A	5/5	1,9 à 3,0 $\times 10^5$	10 %	0/5
E2	5/5	identique à A	69 à 79 %	0/5
E1D	5/5	11 à 34	32 à 88 % (85 à 89 % recalibré)	0/5
E1	0/5	$< 0{,}1$	92,4 à 93,3 %	0/5
E1-O	5/5	jusqu'à 1,9 $\times 10^{16}$	10 %	5/5

Sur c1, E1 reconnaît encore 99,3 à 99,8 % des images à oublier, mais R en reconnaît aussi **98,7 %** (les pantalons qui restent dans D_r suffisent pour les reconnaître). On ne peut donc pas dire qu'E1 n'oublie pas sur cette classe.

Lecture

On en déduit que :

1. faire passer le gradient à travers les statistiques du lot d'oubli **amplifie énormément** l'explosion ($\sim 10^5$ pour A contre 11 à 34 pour E1D) ;
2. normaliser avec les statistiques propres du lot d'oubli **coïncide** avec le franchissement du seuil (E1D le franchit, E1 non) ;
3. des statistiques figées de O explosent aussi, même sur c8 : il n'y a pas d'histoire simple du type « stats fixes contre stats du lot ».

E2 diverge exactement comme A (mêmes poids) mais s'effondre beaucoup moins à l'évaluation : la mise à jour des buffers par le lot d'oubli joue sur l'*évaluation*, pas sur la divergence.

Note sur les gradients. On a aussi regardé après coup les normes de gradient pendant ces runs. Les gradients du lot à garder et du lot à oublier ne se contredisent pas (cosinus ≈ 0), donc ce n'est pas un conflit entre les deux objectifs. Et la norme du gradient explose environ 28 pas (en médiane) *après* que CE_f a dépassé 10 : c'est un symptôme de la divergence, pas quelque chose qui l'annonce.

3 Étape 2 : réplication avec des tests statistiques

On analyse maintenant ces diagnostics avec des tests statistiques. Pour ça on prend de **nouveaux modèles** (graines dérivées autrement) et on écrit le protocole **avant** de lancer : 3 études, 1 141 unités, 6 h sur A100, 9 801 contrôles réussis.

3.1 Plan

— **Étude 1** : Fashion-MNIST, CNN avec BatchNorm, variantes A et C, 5 graines.

— **Étude 2** : le même CNN avec **GroupNorm** au lieu de BatchNorm. GroupNorm normalise chaque image toute seule, sans regarder le lot. 5 graines.

— **Étude 3** : CIFAR-10 (50 000 images couleur 32×32) avec un ResNet-18 à BatchNorm, variantes A et C, 3 graines.

Pour l'étude 1, on reprend la recette de O (15 époques, 55k images, un O par graine). Pour chaque graine on a 11 façons d'oublier (10 classes + stratifié), et pour chacune :

— un modèle de référence R (O réentraîné depuis zéro sans l'ensemble à oublier) ;

— un fine-tuning comme point de comparaison ;

— NegGrad+ avec les variantes A et C, chacune avec 3 poids d'oubli (0,001 ; 0,01 ; 0,05).

Donc $11 \times 2 \times 3 = 66$ runs NegGrad+ par graine, soit 330 runs sur 5 graines.

3.2 La statistique U

Pour chaque run, on calcule S_{moy} (la moyenne des $\log_{10} \text{CE}_f$) : plus il est grand, plus le run est instable. On veut comparer un run concentré au run stratifié **apparié** (même graine, même variante, même poids, même O) :

$$\psi_c = \begin{cases} 1 & \text{si le run concentré est plus instable} \\ 0{,}5 & \text{en cas d'égalité} \\ 0 & \text{sinon} \end{cases}$$

Un run avec un échec numérique est toujours le plus instable (on n'en a pas eu). Puis :

$$U_s = \text{moyenne des } \psi_c \text{ sur les 10 classes} \times 3 \text{ poids} = 30 \text{ comparaisons, pour la graine } s$$

— U dit à quel point les runs concentrés sont plus souvent instables que la référence stratifiée. $U = 0{,}5$ c'est comme si un run concentré était au hasard plus ou moins instable qu'un stratifié, donc pas d'effet ; $U = 1$ veut dire toujours plus instable.

— **Pourquoi par graine** : les 11 runs d'une graine partagent le même O, donc ils ne sont pas indépendants. Les compter séparément gonflerait la précision. On a donc une valeur de U par graine, puis un **test t** sur les graines.

On formule deux hypothèses principales : $U_A - 0{,}5 > 0$ (sous A les runs concentrés sont plus instables) et $U_A - U_C > 0$ (l'effet est plus fort sous A que sous C) ; pour l'étude 2, $U_{GN} - 0{,}5 > 0$.

p-value. C'est la probabilité d'obtenir un résultat *au moins aussi extrême* si l'effet n'existait pas. Ce n'est pas la probabilité que le résultat soit dû au hasard.

$p_{\textbf{Holm}}$**.** Quand on fait plusieurs tests en même temps, le risque qu'au moins un « réussisse » par chance augmente (avec 10 tests à 5 %, c'est environ 40 %). La correction de Holm ajuste les p-values pour garder ce risque global à 5 % (ici, une famille $\{H_1, H_{2a}\}$ par étude).

TOST. « Pas de différence détectée » ne veut pas dire « pas d'effet ». Le TOST montre que l'écart moyen de perte concentré − stratifié (Δ_{moy}, en décades) est **plus petit qu'une marge** fixée à 1 décade : l'intervalle de confiance à 90 % doit être entièrement dans $[-1; 1]$.

3.3 Résultats

Étude	Test	Estimation [IC95]	p_{Holm}	Conclusion
1 (BN)	$U_A - 0{,}5$	0,36 [0,33 ; 0,39] ($U_A = 0{,}86$)	$1{,}7 \times 10^{-5}$	effet
1 (BN)	$U_A - U_C$	0,37 [0,30 ; 0,45]	$1{,}5 \times 10^{-4}$	effet
2 (GN)	$U_{GN} - 0{,}5$	−0,033 [−0,075 ; 0,008]	0,089	rien de détecté
3 (CIFAR)	$U_A - 0{,}5$	0,5 ($U_A = 1$ sur les 3 graines)	–	non calculable
3 (CIFAR)	$U_A - U_C$	0,43 [0,35 ; 0,52]	0,0039	effet

Pour l'étude 3, $U_A = 1$ sur les 3 graines : la variance est nulle, donc le test t n'est pas calculable, mais tous les runs concentrés sont plus instables.

TOST (marge ±1 décade)	Δ_{moy} [IC90]	Équivalent à 0 ?
Étude 1, A	2,39 [2,32 ; 2,47]	non
Étude 1, C	−0,14 [−0,24 ; −0,04]	oui
Étude 2, GroupNorm	−0,14 [−0,20 ; −0,08]	oui
Étude 3, A	2,12 [2,08 ; 2,15]	non
Étude 3, C	0,07 [0,04 ; 0,10]	oui

Lecture

Sous A, les runs concentrés sont presque toujours plus instables (en moyenne 2,39 décades de perte en plus, environ 250 fois plus). L'effet disparaît sous C **et avec GroupNorm**, et le TOST montre qu'il y est négligeable : c'est bien la normalisation **par les statistiques du lot** qui est en cause. Sur Fashion-MNIST à 0,01, A diverge dans 30 runs concentrés sur 50 (24 effondrements), C dans aucun ; et ce sont exactement les six mêmes classes qu'à l'étape 1, avec des modèles tout neufs.

3.4 Particularités de CIFAR-10 / ResNet-18

On entraîne sur 30 époques (lr 0,1 en cosinus) avec des augmentations (recadrage aléatoire avec un padding de 4 et retournement horizontal) ; 348 pas par époque et 490 images à oublier (49 par classe en stratifié). On retrouve les mêmes conclusions qu'à l'étude 1 avec une architecture plus grosse, avec quelques différences :

— sous A, les ensembles concentrés divergent 0/30, 30/30 et 30/30 fois aux poids 0,001, 0,01 et 0,05, mais il n'y a **aucun effondrement** : divergence et effondrement sont deux choses différentes ;

— la perte de précision est beaucoup moins marquée qu'avec le petit CNN, et on la récupère presque entièrement en recalibrant, sauf à 0,05 :

Perte de précision test de A vs O	0,001	0,01	0,05
Buffers natifs	5,6 pts	7,2 pts	28,5 pts (pire run : 43,9 %)
Après recalibration	1,5 pt	1,8 pt	7,7 pts

On peut spéculer que les skip connections limitent la propagation de l'instabilité due à BatchNorm, mais c'est une *hypothèse non testée*.

Attention

Le notebook d'analyse n'a pas été commité avant l'exécution ; 5 et 3 graines, donc des résultats exploratoires d'après le protocole lui-même ; et l'identité des classes et la concentration restent mélangées (toujours les mêmes classes).

4 Étape 3 : quelle partie de BatchNorm ?

On veut maintenant étudier les mécanismes de l'instabilité de BatchNorm, notamment quelle partie l'engendre. On se pose deux questions :

(Q1) Est-ce que faire passer le gradient à travers les statistiques de BatchNorm amplifie l'instabilité ? (garder μ et σ^2 détachés de la backward ou pas) → on compare A et E1D.

(Q2) Est-ce qu'utiliser les stats du lot d'oubli plutôt que les buffers courants amplifie l'instabilité ? → on compare E1D et E1.

Variante	Lot d'oubli normalisé par	Gradient via stats	Met à jour buffers
A	ses propres stats	oui	oui
E2	ses propres stats	oui	non
E1D	ses propres stats	non	non
E1	les buffers courants	non	non

4.1 Plan

On introduit une petite nouveauté : plusieurs tirages par classe. Pour chaque graine :

— 3 tirages de 600 images pour chaque classe (30 ensembles concentrés) ;

— 10 tirages pour le stratifié ;

— donc 40 ensembles à oublier par graine.

Avec 3 poids et 5 graines, ça fait **600 unités** (450 concentrées, 150 stratifiées). Les graines sont dérivées autrement qu'à l'étape 2, donc on a de nouveaux O. A et E2 ont **exactement les mêmes poids** (seuls leurs buffers diffèrent), donc ils forment une seule trajectoire : on a 3 trajectoires par unité (A/E2, E1D, E1), plus FT, R, O et les contrôles, soit **2 445 exécutions** au total (5,39 h).

4.2 Mesures

— **A contre E1D** : on utilise la même méthode S_{moy} sur les 425 pas. C'est comparable parce qu'à poids égaux, A et E1D ont le même forward. On formule H_B : en concentré, A est-il plus instable que E1D ?

— **E1D contre E1** : leurs forwards sont différents (les buffers ne sont pas mis à jour par la même chose), donc on utilise une mesure commune. Tous les 25 pas, on s'arrête et on prend un buffer propre calculé de la même manière pour E1D et E1 (mis à jour seulement par les lots à garder) ; avec ce buffer, on mesure la perte sur les 600 images à oublier. On s'arrête 17 fois ($25 \times 17 = 425$ pas) et on fait la moyenne des $\log_{10} \mathrm{CE}_f$ sur ces 17 points. C'est H_F.

— θ = la part des unités concentrées où la première variante est plus instable ; $\iota = \theta_{\text{conc}} - \theta_{\text{strat}}$ dit si l'effet est plus fort en concentré qu'en stratifié (I_B et I_F). Holm sur les 4 tests, écarts continus en décades, marge d'équivalence ± 1 décade.

4.3 Résultats

Test	θ ou ι	p_{Holm}	Écart continu [IC95]	Conclusion
H_B : A contre E1D	0,82	$3{,}7 \times 10^{-5}$	+0,83 [0,81 ; 0,86]	soutient
H_F : E1D contre E1	0,65	$7{,}5 \times 10^{-4}$	+0,04 [−0,08 ; 0,16]	affaiblit (écart équivalent à 0)
I_B : effet A/E1D plus fort en concentré ?	+0,33	$4{,}0 \times 10^{-4}$	+0,83 [0,81 ; 0,86]	soutient
I_F : effet E1D/E1 plus fort en concentré ?	+0,40	$7{,}5 \times 10^{-4}$	+0,89 [0,70 ; 1,09]	soutient, mais voir ci-dessous

Divergences (sur 150 concentrés, 50 stratifiés)	0,001	0,01	0,05
A, concentré	12	90	150
E1D, concentré	12	89	121
E1, concentré	0	60	150
E1, stratifié	0	0	47
A et E1D, stratifié	0	0	0

Lecture

1. **Q1 : oui.** En concentré, A atteint une perte d'oubli plus élevée qu'E1D dans 82 % des unités, des pertes moyennes environ **7 fois plus grosses** (+0,83 décade), et l'écart grandit avec le poids d'oubli (+0,01, +0,75 puis +1,73 décade). C'est propre à la concentration : en stratifié l'écart continu est quasi nul (I_B).
2. **Q2 : effet faible.** E1D est plus instable qu'E1 dans 65 % des cas, mais l'écart continu est équivalent à 0. Le I_F positif vient surtout du **stratifié à 0,05**, où E1 diverge 47 fois sur 50 ; en concentré, le signe change selon le poids (+0,66 décade à 0,01, −0,61 à 0,05). Donc on ne peut pas dire que l'effet de E1D contre E1 est plus fort en concentré.
3. **Aucune variante n'est un remède** : en concentré à 0,05, A diverge 150/150, E1D 121/150 et E1 150/150.

5 Étape 4 : concentration ou classes difficiles ?

On veut maintenant savoir si le problème vient du fait que certaines classes sont plus difficiles à oublier que d'autres, ou de la concentration elle-même. En effet, jusqu'ici (sauf sur ResNet-18) ce sont toujours les six mêmes classes qui s'effondrent ou divergent : peut-être que « concentré » voulait juste dire « contient une classe difficile ».

5.1 Plan

On prend 10 nouvelles graines (401 à 410) : chacune donne un nouveau O, un nouvel ordre pour les images et une nouvelle initialisation. On reste sur la variante A avec $1 - \alpha = 0{,}01$, et on garde C en variante secondaire. Pour chaque graine on construit 68 ensembles de 600 images :

— 10 ensembles purs : 600 images d'une même classe ;

— 45 paires : 300 + 300 de deux classes différentes ;

— 12 ensembles à 5 classes : 120 par classe ;

— 1 ensemble stratifié.

Donc $68 \times 10 = 680$ runs. On mesure la concentration avec l'entropie des étiquettes $H = \log_2(\text{nombre de classes})$: 0 bit pour une classe, 1 pour deux, 2,32 pour cinq, 3,32 pour dix.

5.2 Mesure : la perte précoce

$$S = \text{moyenne des } \log_{10} \mathrm{CE}_f \text{ sur les 24 premiers pas (0 à 23)}$$

Ce choix a été fixé **à l'avance**, sur d'anciennes traces : aucune divergence n'arrivait avant le pas 24 (le seuil de 10 porte sur CE_f), et S séparait déjà les runs qui allaient diverger des autres (AUC 0,92). On mesure donc la cause *avant* l'explosion, pas la taille de l'explosion.

5.3 Modèle

On prédit ce que donnerait chaque mélange si seules les classes comptaient. Chaque classe c a un effet x_c, et pour un mélange avec des proportions p_c, la prédiction est une moyenne de puissance :

$$M_\lambda = \Big(\sum_c p_c\, x_c^\lambda\Big)^{1/\lambda} \qquad \lambda = 1 : \text{arithmétique}; \quad \lambda \to 0 : \text{géométrique}; \quad \lambda = -1 : \text{harmonique}$$

Au lieu de deviner λ, on le laisse entre -3 et 3. On a donc le modèle suivant :

$$S = \log_{10} M_\lambda(p\,; x) + \gamma \cdot H + \text{bruit}$$

Si l'identité des classes explique tout, $\gamma = 0$; si étaler la demande sur plusieurs classes aide au-delà des classes qu'elle contient, $\gamma < 0$.

```
pour chaque graine s = 1, ..., 10 :
    ajuster (x0...x9, λ, γ) sur ses 68 ensembles par moindres carrés non linéaires
    garder γ_s
statistique t sur les 10 γ_s
p-value par bootstrap paramétrique : 2000 expériences simulées avec γ = 0,
    chacune réajustée ; p = part des t simulés au moins aussi extrêmes que le vrai
```

Le bootstrap ne sert **pas** à estimer x, λ et γ (ça c'est les moindres carrés) : il sert à calibrer le seuil du test sur γ, parce que λ est estimé et que la table de Student n'est alors plus fiable. L'hypothèse nulle est « la concentration n'a aucun effet sur la perte précoce ».

5.4 Résultats

Résultat	**Valeur**
γ	$-0{,}246$ décade par bit $[-0{,}263\,;\,-0{,}228]$
p (bootstrap)	0,001, le plus petit que ce bootstrap peut donner
Graines avec $\gamma < 0$	10 sur 10
λ estimé	entre 0,34 et 1,09
γ sous C (secondaire)	$-0{,}004$, environ 0

Classes dans D_f	**Div. (A)**	**Effondr. (A)**
1	62/100	54/100
2	67/450	22/450
5	0/120	0/120
10	0/10	0/10

Lecture

Donc il y a bien un effet : plus H est petit (plus l'ensemble à oublier est concentré), plus la perte précoce est grande, **au-delà de ce que prédisent les classes qui le composent**. De 1 à 10 classes (3,32 bits), la perte précoce baisse de $0{,}246 \times 3{,}32 \approx 0{,}82$ décade, soit environ **6,6 fois** moins que ce que les classes prédisent. Sous C (BatchNorm en mode eval), l'effet disparaît.

Attention

Un seul poids d'oubli, un seul modèle et un seul jeu de données ; le modèle ne peut pas représenter des interactions propres à certaines paires de classes ; et la mesure porte sur la perte précoce, pas directement sur la divergence ni sur l'oubli.

6 Étape 5 : est-ce l'objectif de NegGrad+ ?

On veut maintenant savoir si l'instabilité vient seulement de BatchNorm, ou si l'objectif de NegGrad+ lui-même est le problème. On compare donc NegGrad+ à une méthode d'oubli dont l'objectif est borné : les **étiquettes aléatoires** (RL, random labels). Pour chaque image à oublier, on tire une mauvaise étiquette parmi les 9 autres classes (fixée une fois pour toutes par graine et par ensemble), et on entraîne le modèle vers cette mauvaise étiquette :

$$L = \alpha\, \mathrm{CE}_r + (1-\alpha)\, \mathrm{CE}(x_f,\ \text{mauvaise étiquette})$$

La différence avec NegGrad+, c'est que cette perte a un **plancher**. Prenons une bottine à oublier, à qui RL a donné l'étiquette « basket » :

NegGrad+ : faire monter $-\log p(\text{bottine})$		**RL : faire descendre** $-\log p(\text{basket})$	
p(bottine)	perte	p(basket)	perte
0,99	0,01	0,01	4,6
0,1	2,3	0,5	0,69
10^{-4}	9,2	0,99	0,01
$\to 0$	$\to +\infty$	$\to 1$	$\to 0$

Avec NegGrad+, il n'y a pas de point d'arrivée : on peut toujours rapprocher p(bottine) de 0, donc la perte monte sans fin et le gradient continue de pousser (seul le clipping le freine). Avec RL, la perte ne peut pas descendre sous 0 : quand le modèle dit déjà « basket », le terme de cette image et son gradient deviennent tout petits, donc **la poussée s'éteint d'elle-même**. En gros, RL c'est « va jusqu'au mur et arrête-toi », NegGrad+ c'est « éloigne-toi du mur le plus loin possible ».

Note : ça ne veut pas dire que p(basket) monte à chaque pas. Chaque pas ne voit qu'un lot de 64 images, le pas a une taille finie (momentum, clipping), et surtout le terme « garder » (99 % de la perte) tire dans l'autre sens, puisque D_r contient d'autres bottines étiquetées « bottine ». Le plancher dit seulement que la perte est bornée par le bas et que la poussée ne peut pas s'emballer. (On n'a pas suivi p(basket) pas à pas : on a mesuré les gradients, les divergences et la précision.)

Tout le reste est identique : même O, mêmes ensembles à oublier, mêmes lots dans le même ordre, même traitement BatchNorm (variante A), même lr et même clipping à 5. Comme ça, la seule chose qui change c'est l'objectif : si NegGrad+ explose et pas RL, ça ne peut venir que de là.

6.1 Premier essai : la charge d'écrêtage

On a d'abord mesuré la **charge d'écrêtage** B = la part des 425 pas où la norme du gradient dépasse 5 et doit être clippée. $B = 0$: jamais de gradient trop grand ; $B = 0{,}5$: un pas sur deux.

Pour chaque méthode, l'effet de la concentration est $\Delta = B_{\text{concentré}} - B_{\text{stratifié}}$, et on compare les deux méthodes avec

$$\theta_B = \Delta_{\text{NegGrad+}} - \Delta_{\text{RL}}$$

(10 graines dont 5 reprises de l'étape 2, 782 runs). On trouve $\theta_B = 0{,}30$ [0,29 ; 0,32] : en concentré, NegGrad+ doit clipper environ 30 % de pas en plus que RL.

Attention

Sous RL, B est collé à 0 (4 pas clippés sur 330 runs), en concentré comme en stratifié, donc $\Delta_{\text{RL}} \approx 0$. Mais ça ne prouve pas que RL n'est pas touché : si la concentration fait passer la norme du gradient de 1 à 2, ça reste sous le seuil de 5 et B ne voit rien. C'est un **effet plancher**, comme un détecteur de fièvre qui ne sonne qu'au-dessus de 40 °C. Il fallait donc une mesure continue, sans seuil.

6.2 Deuxième essai : la taille typique des gradients $\bar{G}$

On prend 10 nouvelles graines (101 à 110), donc de nouveaux modèles, rien de réutilisé. À chaque pas on mesure la norme du gradient de toute la perte avant le clipping, et :

$$\bar{G} = \text{moyenne sur les 425 pas de } \log_{10}(\text{norme du gradient})$$

$\bar{G}$ mesure à quel point l'optimiseur est poussé fort ; ce n'est pas une mesure d'instabilité en soi. $+1$ en $\bar{G}$ veut dire des gradients 10 fois plus grands, $+0{,}1$ environ 26 % plus grands. Pour chaque graine et chaque méthode :

$$\text{décalage} = \overline{\bar{G}}_{\text{10 purs}} - \bar{G}_{\text{stratifié}}, \qquad \theta = \text{décalage}_{\text{NegGrad+}} - \text{décalage}_{\text{RL}}$$

Ça donne une valeur de θ par graine, donc 10 valeurs, et on fait un test t dessus au poids 0,01 (test principal) ; les poids 0,001 et 0,05 sont en secondaire avec Holm. Avant de lancer, on a simulé l'analyse sur des données sans aucun effet pour vérifier qu'elle ne crie pas au loup : entre 4,4 % et 5,9 % de faux positifs, donc OK.

Résultat (poids 0,01)	**Valeur**
Décalage sous NegGrad+	0,705 décade (gradients ∼5 fois plus grands en concentré)
Décalage sous RL	0,053 décade (gradients ∼13 % plus grands)
θ	0,652 décade [0,607 ; 0,696], $t = 33{,}3$
Graines avec $\theta > 0$	10 sur 10

Poids d'oubli $1-\alpha$	**0,001**	**0,01**	**0,05**
θ	−0,015	0,652	2,644
Perte de précision test en concentré, NegGrad+	1,3 pt	46,8 pts	82,9 pts
Perte de précision test en concentré, RL	1,0 pt	2,0 pts	2,9 pts

Lecture

Donc en gros, la concentration perturbe les deux méthodes, mais NegGrad+ beaucoup plus : c'est l'objectif sans plafond qui transforme la petite perturbation due à BatchNorm en explosion. À 0,001, la poussée est trop faible pour que ça compte. Attention : ça ne montre pas que RL est instable (son petit décalage ne s'accompagne jamais d'une divergence), ni quelle partie du réseau est responsable.

7 Étape 6 : des remèdes à l'instabilité

On sait maintenant que le problème vient de BatchNorm sur des lots mono-classe et de l'objectif sans plafond de NegGrad+. Du coup on teste trois remèdes, chacun visant un suspect différent.

7.1 Comment on mesure les dégâts

Il faut d'abord une mesure de « à quel point le modèle est abîmé » :

$$U = \text{précision de } O \text{ sur } D_r - \text{précision du modèle désappris sur } D_r$$
$$\text{pénalité} = \text{moyenne de } U \text{ sur les 10 concentrés} - U \text{ du stratifié} \quad \text{(par graine)}$$

Les deux modèles sont recalibrés sur D_r avant de mesurer U, pour mesurer les dégâts dans les poids et pas la pollution des buffers. L'effet d'un remède, c'est I = pénalité de NegGrad+ normal − pénalité avec le remède.

Attention

Une petite pénalité peut juste vouloir dire qu'on n'a presque rien fait : un modèle qui n'oublie rien n'abîme rien. Donc on regarde toujours à côté la **force** E : de combien la proba de la bonne classe sur D_f a baissé par rapport à O (en décades, buffers recalibrés). Et la cible, c'est R : en concentré, R reconnaît encore ∼93 % des images à oublier (il a appris la classe avec les autres images), donc le but n'est pas 0 %.

7.2 Remède principal : le réglage par condition

L'idée, c'est que peut-être NegGrad+ n'a rien de fragile et que le réglage par défaut (lr = 0,01 et $1 - \alpha = 0{,}01$) est juste mauvais pour les ensembles concentrés. Donc on cherche le meilleur réglage pour chaque classe, parmi 20 configurations :

$$\text{lr} \in \{0{,}001; 0{,}003; 0{,}01; 0{,}03\} \times 1 - \alpha \in \{0{,}001; 0{,}003; 0{,}01; 0{,}03; 0{,}05\}$$

On règle sur 3 graines (447 à 449) avec une règle fixée avant :

1. **stable** = aucune divergence ni effondrement sur les 3 graines ;
2. **faisable** = stable *et* force $E \geq$ celle de R ;
3. parmi les faisables, on prend la meilleure précision de validation (recalibrée) ;
4. s'il n'y a aucune config faisable : parmi les stables, celle qui oublie le plus.

Ensuite on évalue sur 12 autres graines (450 à 461) qui n'ont pas servi au réglage ; sinon on mesurerait juste la chance d'avoir trouvé une config qui marche sur ces 3 graines-là. Les 12 graines viennent d'une simulation de puissance. On compare NegGrad+ réglé et NegGrad+ par défaut, avec un test t sur la pénalité et un TOST avec une marge de ±2 points.

	NegGrad+ par défaut	**NegGrad+ réglé**
Pénalité de concentration	14,2 pts [13,0 ; 15,5]	1,07 pt [0,88 ; 1,27]
TOST (marge ±2 pts)	–	équivalente à 0
Divergences en concentré	71 / 120	2 / 120
Effondrements en concentré	65 / 120	–

Sur le TOST : l'intervalle [0,88 ; 1,27] ne contient pas 0, donc il reste une petite pénalité ; mais il est entièrement sous 2, donc elle est petite. Les deux sont vrais en même temps.

Lecture

Le réglage enlève presque toute la pénalité. Seulement, il n'y a que 3 classes sur 10 (c4, c8, c9) qui ont une config à la fois stable et aussi forte que R. Le stratifié est stable avec les 20 configs, les ensembles concentrés avec 1 à 16 selon la classe. Pour les 7 autres classes, la règle a pris la config stable la plus forte, qui oublie moins que R. En évaluation, la force de R n'est atteinte que dans 20 % des runs concentrés (67 % en stratifié).
Donc en gros : **pour rester stable en concentré, il faut pousser moins fort que ce qu'il faudrait pour oublier autant que** R.

7.3 Les deux autres remèdes

— **Cible figée** : au lieu de faire monter la perte à l'infini, on vise une réponse précise. Pour chaque image à oublier, la cible est la sortie de O où on a enlevé la vraie classe (on la masque puis on renormalise). C'est un objectif borné comme RL, mais plus naturel (une bottine ressemble à une basket, pas à un sac). Sur 12 graines (421 à 432), la pénalité baisse de 16,2 points [14,1 ; 18,4] par rapport à NegGrad+, et il y a 0 effondrement contre 70/120. Mais le modèle reconnaît encore ∼97 % des images à oublier, et il n'atteint sa cible qu'en mode train, dans des lots mono-classe : une fois recalibré, le changement a disparu.

— **RD64** : on garde NegGrad+, mais le lot à oublier est normalisé avec les stats de 64 images du lot à garder (qui est mélangé), détachées du gradient et sans mise à jour des buffers. Sur 14 graines (433 à 446), la pénalité baisse de 13,7 points [12,6 ; 14,7] sur les 14 graines, et les divergences passent de 84/140 à 0/140. Mais comme on change trois choses en même temps, on ne sait pas laquelle fait le travail.

Précision sur les images à oublier (concentré, recalibré)	
NegGrad+ normal (A) : il va trop loin, il casse	80,2 %
R : la cible	93,1 %
NegGrad+ avec RD64	96,7 %
Cible figée	∼97 %
Fine-tuning (ne fait presque rien)	98,5 %

Lecture

Donc en gros, les trois remèdes évitent la casse, mais surtout parce qu'ils oublient moins : ils se rapprochent du fine-tuning, pas de R. Ce sont des remèdes **à l'instabilité**, pas des remèdes **à l'oubli** : ils éliminent les divergences et la pénalité, mais aucun n'oublie autant que R. Rendre NegGrad+ à la fois stable et aussi efficace que le réentraînement sur des ensembles concentrés reste un problème ouvert.

8 Limites et perspectives

— On ne mesure jamais un vrai effacement des données : il n'y a qu'une attaque d'appartenance, exploratoire, sur Tiny ImageNet.

— BatchNorm est le suspect principal, mais on n'a pas isolé le mécanisme : la plupart des variantes changent plusieurs choses à la fois.

— C'est surtout un petit CNN sur Fashion-MNIST, avec 3 à 14 graines. Sur Tiny ImageNet (200 classes, ResNet-18), l'effet va dans le même sens mais il est beaucoup plus faible ($\theta = 0{,}029$ décade).

— Les études ne sont pas des réplications indépendantes (même découpage des données).

— Perspectives : lancer l'étude PCGrad déjà écrite (jamais exécutée), remplacer BatchNorm par GroupNorm ou LayerNorm dans toute la chaîne, mesurer vraiment l'oubli des remèdes, et essayer de prédire à l'avance les demandes d'oubli risquées.